

The pyflightstream guides, guide 3 of 9, numbered 1 to 9

Guide 3: From the GUI to pyfs

Every step of a GUI session, and the key that does it in the workspace

Written for pyflightstream 0.33.0.

Source: the package page “From the GUI to pyfs” [1].

pyflightstream is the work of Geovana Neves: <https://github.com/nevesgeovana/pyflightstream>, concept DOI 10.5281/zenodo.21482924.

Copyright (c) 2026 Geovana Neves. Licensed under CC BY 4.0

(<https://creativecommons.org/licenses/by/4.0/>). See LICENSE-AND-AUTHORSHIP.md.

How to read this deck

- A GUI session is a sequence of steps: load the geometry, mark where its wakes leave it, place frames and set things turning, state the flight condition and the solver, run, export. The deck takes them in that order, one table per stage.
- **In pyfs** says which file the key lives in: a *row key* (a matrix column or the VAR_NAMES_VALUES cell), a *setup key*, a *pproc* table, the *reference*, or the *sidecar* `<stem>.boundaries.toml` beside the geometry.
- **Verified on:** the builds on which the command database records a probe run of every command the key writes. *none*: no probe run, the command is still documented by its build's manual. *no command*: the step writes none.
- **not yet:** no key of its own. The raw route still takes it (last part).

Source: docs/gui-to-pyfs.md, How to read a line.

Stage	Key	Not yet
Geometry and mesh	7	6
Boundary conditions	14	0
Frames, motion, actuators	11	5
Flight conditions, solver	24	0
The run	11	0
Basic post	25	5
108 steps	92	16

Contents

Geometry and mesh

Boundary conditions

Frames, motion and actuators

Flight conditions and solver

The run

Basic post

The raw route

Part 1

Geometry and mesh

Geometry and mesh: what a key takes (1/2)

In the GUI	In pyfs	Verified on
Open a saved simulation	row key <code>GEOMETRY</code> naming a <code>.fsm</code> ; setup key <code>load_solver_initialization</code> loads the solver state it was saved with	26.100 to 26.124
See the boundaries a saved simulation carries, in its order	sidecar key <code>boundaries</code> , which <code>pyfs-matrix</code> inventory writes from the file	none
Start a new simulation and import a mesh file in its length unit	row key <code>GEOMETRY</code> naming an <code>.obj</code> or <code>.stl</code> ; sidecar table <code>[import]</code> with its units, and sidecar key <code>boundaries</code> , the file's surface names in its order, which the plan writes from an OBJ's groups; setup key <code>simulation_length_unit</code> states the simulation's own length unit	26.101 to 26.124, except one

Source: `docs/gui-to-pyfs.md`, Geometry and mesh.

Geometry and mesh: what a key takes (2/2)

In the GUI	In pyfs	Verified on
Scale, mirror, rename, move or turn a surface of the imported mesh	sidecar table <code>[[import.operations]]</code> , one per operation	none
Turn part of the geometry for one row: a flap, a blade's pitch	row key <code>ROTATE</code>	none
Move part of the geometry for one row	row key <code>TRANSLATE</code>	none
Name the configuration you loaded	row key <code>CONFIGURATION</code> , a label the script carries as a comment on its first line	none

Source: `docs/gui-to-pyfs.md`, Geometry and mesh.

Geometry and mesh: not yet

- Import CAD and mesh it (`IMPORT_CAD`, `CONVERT_CAD_TO_MESH`).
- Build a wing, a fuselage or a body of revolution from cross-section curves (`CCS_IMPORT`, `CAD_CREATE_WING_MESH_FROM_CCS`, `CAD_CREATE_FUSELAGE_MESH_FROM_CCS`, `CAD_CREATE_REVOLVE_MESH_FROM_CCS`).
- Wrap or unite meshes into one closed surface (`WRAPPER_EXECUTE`, `BOOLEAN_UNITE_MESH`).
- Repair a surface: delete, combine, invert, cut by a plane, fill holes (`SURFACE_DELETE`, `SURFACE_COMBINE`, `SURFACE_INVERT`, `SURFACE_CUT_BY_PLANE`, `SURFACE_AUTO_HOLE_FILL`, `DELETE_DEGENERATE_FACES`).
- Copy a surface around an axis or along a line (`SURFACE_CIRCULAR_COPY_PASTE`, `SURFACE_LINEAR_COPY_PASTE`).
- Export the surface mesh (`EXPORT_SURFACE_MESH`).

Takeaway *A step that is more than a few lines, such as a geometry built from CAD, is done once in the GUI and saved as a .fsm: GUI once, script everything after.*

Source: docs/gui-to-pyfs.md, Geometry and mesh.

Part 2

Boundary conditions

Boundary conditions: what a key takes (1/3)

In the GUI	In pyfs	Verified on
Mark the trailing edges from a file of edge points	sidecar table [trailing_edges] with file, and type and tolerance where not the default	26.124
Detect the trailing edges over the whole mesh	sidecar table [trailing_edges] with detect = "auto"	26.101 to 26.122
Detect the trailing edges of the surfaces you name, above a sweep angle	sidecar table [trailing_edges] with detect = { surfaces = [...], sweep_angle = ... }	none
Mark the wake-termination nodes, over the whole mesh or by surface	sidecar table [wake_termination]	none
Make the boundaries you name base regions	row key BASE_REGIONS, or pproc key base_regions, naming the base and not the body that carries it	none
Detect the base regions over the whole mesh	sidecar table [base_regions] with detect = "auto"	none
Apply or skip the trailing edges, wake-termination nodes and base regions the geometry declares	setup keys apply_trailing_edges, apply_wake_termination and apply_base_regions; false skips the redefinition and never clears saved state	none

Source: docs/gui-to-pyfs.md, Boundary conditions.

Boundary conditions: what a key takes (2/3)

In the GUI	In pyfs	Verified on
Change one trailing edge's type: relaxed, jet outflow, vortex shedding	setup key <code>trailing_edge_types</code> , each trailing-edge index with its type	26.121 to 26.124
Mark wake-termination nodes by hand, or stop a trailing edge's wake	setup keys <code>mark_wake_termination_nodes</code> and <code>disabled_wake_trailing_edges</code>	26.121 to 26.124, except one
Shed wakes from the leading edges of a surface	setup key <code>leading_edge_wake_boundaries</code>	none
Create, remesh, delete or bend a base region, set its pressure, select its faces or mark its trailing or outflow edges	setup key <code>base_region_operations</code> , applied in the written order, and setup key <code>base_region_bending_angle_deg</code> for the detection angle	none
Add an inlet or an outlet	the geometry's <code>[ports]</code> identifies the surfaces; setup table <code>[[ports]]</code> selects each one's role, and the matrix row supplies its velocity and profile variables. Creating ports on a saved simulation stays refused while its existing port indices are unknown	none

Source: `docs/gui-to-pyfs.md`, Boundary conditions.

Boundary conditions: what a key takes (3/3)

In the GUI	In pyfs	Verified on
Delete an existing inlet or outlet	setup keys <code>delete_inlets</code> and <code>delete_outlets</code> , port indices in the written order	none
Set the edge bluntness angles and the vertex merge tolerance	setup keys <code>geometric_edge_bluntness_angle_deg</code> and <code>vertex_merge_tolerance_m</code> ; the trailing-edge bluntness angle through setup table <code>[[flags]]</code> , which gives a one-value command a word of your own, or setup table <code>[[raw]]</code>	none

The trailing edge by file is the default route

Detection is an angle criterion, and a twisted blade's trailing edge is not a crease. The file names exactly the edges; the run checks the solver's count of imported edges against it.

Source: `docs/gui-to-pyfs.md`, Boundary conditions.

Part 3

Frames, motion and actuators

Frames, motion and actuators: what a key takes (1/2)

In the GUI	In pyfs	Verified on
Create a coordinate system	reference table <code>[[frames]]</code>	26.120 to 26.124
Create a rotor's hub and blade frames	reference block <code>kind = "rotor":</code> its hub, axis and <code>families_blades</code>	26.120 to 26.124
Turn a rotor: a rotary motion about its axis, at its speed, moving its blades and frames	row key <code>RPM</code> or <code>ADVANCE_RATIO</code> ; the rotor's reference block gives its hub, axis, hand (<code>rpm_sign</code>) and blades, and a row stating <code>RPM_SIGN</code> is refused	none
Hold the blades still and turn the free stream about the shaft	<code>WORKFLOW</code> <code>qsteady_rotor</code> : a sector where the row states periodic symmetry, else the whole wheel; <code>PASSAGE_POSITIONS</code> for a wheel at incidence or in a custom inflow	none
Turn several rotors in one run	row keys <code>MOTIONS</code> and <code>CLOCK_MOTION</code> : one record per rotor, and the rotor whose turn sets the time step	none

Source: `docs/gui-to-pyfs.md`, Frames, motion and actuators.

Frames, motion and actuators: what a key takes (2/2)

In the GUI	In pyfs	Verified on
Choose the direction a rotor's relaxed wake sheds	row key ROTOR_SHEDDING, refused whatever its value: no workflow command applies the direction, so a row stating it is refused rather than run with a wake it did not ask for	none
Turn the free stream: a roll, pitch or yaw rate	row keys roll_rate, pitch_rate and yaw_rate in the FLIGHT_CONDITION cell; reference table [body_axes]	none
Create an actuator disc: its axis, radius, speed and swirl	reference block kind = "actuator"; row keys ACTUATOR and ACTUATOR_RPM	26.121 to 26.124, except one
Load the disc by its thrust	row key ACTUATOR_THRUST	none
Load the disc by a radial profile	row key PROFILE, a file of inputs/profiles/	none
Rename or delete a disc	setup key actuator_operations, each naming one disc by its exact current name	26.120 to 26.124

Source: docs/gui-to-pyfs.md, Frames, motion and actuators.

Frames, motion and actuators: not yet

- Disable a disc, or change its wake (`DISABLE_ACTUATOR`, `SET_ACTUATOR_WAKE_TYPE`).
- Duplicate, mirror or delete a coordinate system (`DUPLICATE_COORDINATE_SYSTEM`, `MIRROR_COORDINATE_SYSTEM`, `DELETE_COORDINATE_SYSTEM`).
- A motion of six degrees of freedom, with its mass, forces and gravity (`CREATE_NEW_MOTION_6DOF`, `SET_MOTION_MASS_PROPERTIES`, `CREATE_NEW_6DOF_EXTERNAL_FORCE`, `SET_MOTION_GRAVITY`).
- A prescribed motion: a velocity, an acceleration, or a table in time (`CREATE_NEW_MOTION_EUCLIDEAN`, `SET_MOTION_VELOCITY`, `SET_MOTION_ACCELERATION`, `CREATE_NEW_MOTION_CUSTOM`, `SET_MOTION_CUSTOM_TABLE`).
- Start a motion after the run has started (`SET_MOTION_START_TIME`).

The rates' sense is measured (26.124)

Roll and yaw take the sign of their body axis: `roll_rate:40` writes `ROTATION <frame> X -6.667 [2]`.

Source: `docs/gui-to-pyfs.md`, Frames, motion and actuators.

Part 4

Flight conditions and solver

Flight conditions and solver: what a key takes (1/6)

In the GUI	In pyfs	Verified on
Set the free stream: its speed, angle of attack and sideslip	row keys ALPHA and BETA in the FLIGHT_CONDITION cell, whose MACH or TASmps gives the speed; row key VELOCITY from Python alone	26.120 to 26.124, except one
Set a custom free stream: import a velocity field over the YZ plane from a file	row key FREESTREAM, the stem of a file of inputs/freestreams/, a .txt in the STRUCTURED form or a .dat in the UNSTRUCTURED form, in m and m/s; the field is the flow's direction, so the row's angle of attack and sideslip are 0	none
Sweep the angle of attack or the sideslip	ALPHA:sweep or BETA:sweep in the FLIGHT_CONDITION cell, with the SWEEP_VALUES column: one script; every point starts cold, clearing the solution before it, and row key COLD_START set to false keeps the earlier warm start from the last point	26.120 to 26.124
Set the fluid: density, pressure, temperature, viscosity, speed of sound	the FLIGHT_CONDITION cell, or the setup's [flight_condition] table	26.101 to 26.124

Source: docs/gui-to-pyfs.md, Flight conditions and solver.

Flight conditions and solver: what a key takes (2/6)

In the GUI	In pyfs	Verified on
Set the reference area, length and velocity	reference lengths; setup key <code>reference_velocity_m_per_s</code>	26.101 to 26.124
Give the free stream as a Mach number, set the reference Mach number or the speed of sound, or switch the reference velocity off	setup keys <code>freestream_input</code> , <code>reference_mach</code> and <code>disable_reference_velocity</code> ; setup key <code>sonic_velocity_m_per_s</code> is refused on every build, since no build records <code>SONIC_VELOCITY</code> and the sound speed follows from temperature and specific-heat ratio	26.120 to 26.124, except two
State the units of a custom free-stream file	row key <code>FREESTREAM_UNITS</code> : SI for metres and metres per second, which the package scales once into the simulation's unit, or <code>NATIVE</code> for a file already in it	none
Choose steady or unsteady, the time step and the number of steps	the <code>WORKFLOW</code> column: <code>steady</code> , <code>unsteady</code> , <code>unsteady_rotor</code> or <code>qsteady_rotor</code> ; row keys <code>DELTA_TIME</code> and <code>TIME_ITERATIONS</code> , or <code>DELTA_THETA</code> and <code>REVOLUTIONS</code> on a rotor row	26.100 to 26.124
Set the iterations and the convergence threshold	setup keys <code>iterations</code> , <code>convergence</code> , <code>forced_iterations</code> and <code>convergence_iterations</code>	26.120 to 26.124

Source: `docs/gui-to-pyfs.md`, Flight conditions and solver.

Flight conditions and solver: what a key takes (3/6)

In the GUI	In pyfs	Verified on
Set the number of processors	row key NCPUS, or setup key max_threads	26.120 to 26.124
Choose the boundary layer and its coupling	setup keys boundary_layer and viscous_coupling	26.120 to 26.124
Switch the viscous coupling on at a time step of an unsteady run	setup key unsteady_viscous_coupling_iteration, refused on 26.124, which does not answer the command	none
Initialise the solver: the compressibility model, wall-collision avoidance, a mirror plane or periodic copies	setup keys solver_model and wall_collision_avoidance; row keys SYMMETRY and PERIODIC_COPIES; setup key legacy_solver_model, the older model command, refused on every build: only 25.000 documents SET_SOLVER_MODEL and no workflow writes that edition's INITIALIZE_SOLVER	26.101 to 26.124, except one
Remove a saved initialization, mark proximal boundaries, set the automatic trailing-edge and wake-node physics, or delete transition trips	setup keys remove_initialization, proximal_boundaries, physics_auto_trailing_edges, physics_auto_wake_nodes and delete_transition_trips	26.121 to 26.124, except two

Source: docs/gui-to-pyfs.md, Flight conditions and solver.

Flight conditions and solver: what a key takes (4/6)

In the GUI	In pyfs	Verified on
Advanced settings: far-field layers, the mesh-induced wake velocity, wake-on-wake induction, a further wake relaxation	setup keys <code>farfield_layers</code> , <code>mesh_induced_wake_velocity</code> , <code>wake_on_wake_induction</code> and <code>additional_wake_relaxation</code>	26.120, 26.122, 26.123
Advanced settings: the minimum pressure coefficient, stabilisation, unsteady pressure and Kutta condition, Reynolds-averaged drag	setup keys <code>minimum_cp</code> , <code>solver_stabilization</code> , <code>unsteady_pressure_and_kutta</code> and <code>reynolds_averaged_drag</code>	26.120, 26.122, 26.123
Advanced settings: wake relaxation, decay and agglomeration, and jet wakes	setup keys <code>wake_relaxation</code> , <code>wake_numerical_relaxation</code> , <code>wake_decay_constant_per_m</code> , <code>wake_streamwise_agglomeration</code> , <code>jet_wake_decay_normalized_length</code> and <code>jet_wake_filaments_grid_induction</code>	none
Advanced settings: where the wake ends	setup key <code>wake_termination_revolutions</code> or <code>wake_termination_steps</code>	none

Source: `docs/gui-to-pyfs.md`, Flight conditions and solver.

Flight conditions and solver: what a key takes (5/6)

In the GUI	In pyfs	Verified on
Advanced settings: the lift and separation models	setup keys <code>kutta_joukowski_lift</code> , <code>vorticity_lift_model</code> , <code>laminar_separation</code> , <code>adverse_gradient_boundary_layer</code> , <code>vortex_ring_normalization</code> and <code>axial_separation_families</code>	none
Advanced settings: rotor-induced velocities, field-grid refinement, the aeroelastic interpolation	setup keys <code>print_rotor_induced_velocities</code> , <code>rotor_induced_velocity_blending</code> , <code>adaptive_field_grid_refinement</code> and <code>aeroelastic_rbf_type</code>	none
Create or delete a separation model: bulk, airfoil, axial vortex, cylindrical, Stratford	setup keys <code>bulk_separation</code> , <code>airfoil_separation</code> , <code>axial_vortex_separation</code> , <code>cylindrical_bulk_separation</code> , <code>stratford_bulk_separation</code> and <code>delete_separations</code>	none
Set the Valarezo and crossflow separation criteria	setup keys <code>valarezo_criterion</code> , <code>valarezo_separation_boundaries</code> , <code>crossflow_separation_boundaries</code> , <code>crossflow_separation_diameter</code> , <code>crossflow_separation_mean_diameter</code> and <code>crossflow_separation_axisymmetric</code>	none

Source: `docs/gui-to-pyfs.md`, Flight conditions and solver.

Flight conditions and solver: what a key takes (6/6)

In the GUI	In pyfs	Verified on
Mark thin or viscous-excluded boundaries, or set a surface roughness	setup keys <code>thin_boundaries</code> , <code>viscous_excluded</code> and <code>surface_roughness</code>	26.121 to 26.124, except two
Couple the blade's structure to the aerodynamics (FSI)	row key <code>FSI</code> , naming a structural configuration of <code>inputs/fsi/</code> , and fourteen row keys <code>FSI_<PROPERTY>_FACTOR</code> , each scaling one structural property; allowed on <code>steady</code> , <code>qsteady_rotor</code> sector and <code>unsteady</code> rows, refused on <code>unsteady_rotor</code> rows (Guide 8)	none

The custom field is the flow's direction

On 26.124 the angle of attack does not turn a custom field [2]: incidence goes into `vy` and `vz`, and a nonzero `ALPHA` or `BETA` beside `FREESTREAM` is refused at plan.

Source: `docs/gui-to-pyfs.md`, Flight conditions and solver.

Part 5

The run

The run: what a key takes (1/2)

In the GUI	In pyfs	Verified on
Start the solver	<code>pyfs-matrix plan</code> , then <code>pyfs-matrix run</code>	26.101 to 26.124
Close FlightStream when the script ends	every run type	26.100 to 26.124
Choose the build	the <code>FS_BUILD</code> column	none
Run on this machine	<code>pyfs-matrix run</code> ; on Linux with an HPC profile, <code>pyfs-matrix run --local</code>	none
Submit to a cluster	<code>pyfs-matrix run</code> on Linux with an HPC profile under <code>inputs/hpc/</code> , then <code>pyfs-matrix collect</code> ; row keys <code>NCPUS</code> and <code>WALLTIME</code>	none
Stop an unsteady run before its wall clock does, so it can be continued	row key <code>WALLTIME</code> , with its unit; setup key <code>walltime_margin_s</code>	none
Continue an unsteady run where it stopped	row key <code>RESTART: {FINISH_PENDING}, {ADDITIONAL_ITERS=n}</code> or <code>{ADDITIONAL_REVS=n}</code>	26.100 to 26.124

Source: `docs/gui-to-pyfs.md`, The run.

The run: what a key takes (2/2)

In the GUI	In pyfs	Verified on
Limit one point's wall clock on this machine	setup key timeout_s, which pyfs enforces around the solver process	none
Run only the points not run yet	pyfs-matrix run --resume	none
Redo a point already run	pyfs-matrix run --force-rerun <point>	none
Skip or refuse a family the mesh does not carry	--ignore-missing-families of plan and run, which reaches each case as row key IGNORE_MISSING_FAMILIES	none

Takeaway Every GUI step of the run has a key: the run is the one stage with no not yet.

Source: docs/gui-to-pyfs.md, The run.

Part 6

Basic post

Basic post: what a key takes (1/5)

In the GUI	In pyfs	Verified on
Export the loads table	<code>pproc table [exports]: loads, always on</code>	26.101 to 26.124
Save the solved simulation	<code>pproc table [exports]: simulation, always on</code>	26.100 to 26.124
Place the moment reference point and take the loads about it	<code>reference table [moment_point]</code> , which creates the frame MRP	26.121 to 26.124
Report the loads of the meshed half or sector, or of the whole	row key <code>SYMMETRY_LOADS</code> , or setup key <code>symmetry_loads</code>	26.120, 26.122, 26.123
Take the induced drag from the vorticity of the surfaces you name	setup key <code>vorticity_drag_families</code> ; setup key <code>clear_vorticity_drag_boundaries</code> clears the selection after a steady solve	26.121 to 26.124, except one
Loads in newtons rather than coefficients, inviscid loads, or boundaries left out of the loads (steady rows)	setup keys <code>load_units</code> , <code>inviscid_loads</code> and <code>analysis_families</code>	26.120, 26.122, 26.123, except one

Source: `docs/gui-to-pyfs.md`, Basic post.

Basic post: what a key takes (2/5)

In the GUI	In pyfs	Verified on
Print more significant digits	setup key <code>significant_digits</code>	none
Export the surface solution as Tecplot, VTK or CSV, with the VTK variables you choose	pproc table [exports]: <code>tecplot</code> (on unless switched off), <code>vtk</code> and <code>csv</code> (off unless switched on); pproc key <code>vtk_variables</code> . The package preserves VTK cell values; pproc key <code>singularity_strength</code> (off unless set to true) adds same-step native nodal <code>Singularity_strength</code> after a unique geometry/topology match	26.120 to 26.124, except one
Export the force distribution, panel by panel	pproc table [exports]: <code>force_distributions</code> , off unless switched on	26.120 to 26.124
Average the surface solution over the last steps of an unsteady run	pproc table [time_averaging]: the row exports its surface at every step of the window and <code>pyfs-matrix post</code> averages them, cell by cell	none

Source: docs/gui-to-pyfs.md, Basic post.

Basic post: what a key takes (3/5)

In the GUI	In pyfs	Verified on
Read boundary-layer integrals along configured cuts	<code>pproc [products]: boundary_layer_integrals = true</code> ; original VTK cell values at recorded section cuts. A fixed-frame native comparison exists on 26.124; other conventions need their own evidence	none
Write sampled velocity fields as VTK, Tecplot or reusable inflow	<code>pproc [[probes]]: field_formats</code> and <code>reusable_inflow</code> ; volume sections use the same probe sampling route. Evidence is specific to the export, the build and the unit; the guide states it	none
Cut surface sections and export their Cp and sectional loads	<code>pproc table [sections]</code>	26.120 to 26.124, except one
Sample a flow-field plane and export velocity	<code>pproc table [volume_section]</code> ; steady probes or unsteady/rotor fluid plots become package-written point fields; evidence is specific to the export, the build and the unit	26.124, except one

Source: docs/gui-to-pyfs.md, Basic post.

Basic post: what a key takes (4/5)

In the GUI	In pyfs	Verified on
Probe the flow at points and along lines (steady rows)	<code>pproc table [[probes]]</code>	26.101 to 26.124, except one
Probe the flow through an unsteady run	<code>pproc table [[probes]], sampled as fluid plots</code>	26.124
Plot the forces through an unsteady run	<code>pproc table [plots]</code>	26.124
Save residual, load and final section Cp plots	<code>pproc table [exports]: plot_residuals, plot_loads and plot_sections_cp; section Cp is exported once after the march, never at every STEP</code>	26.124
Export the solver log	<code>pproc table [exports]: log; row key EXPORT_LOG, which the HPC profile's [log] table writes, and row key LOG_OUTPUT from Python alone</code>	26.100 to 26.124
Export at every step of an unsteady run, from a step on	<code>row keys EXPORT_UNSTEADY_AFTER_ITER and EXPORT_UNSTEADY_AFTER_REV</code>	none
Sum the loads of groups of surfaces into polars	<code>pproc tables [groups] and [products]</code>	none

Source: `docs/gui-to-pyfs.md`, Basic post.

Basic post: what a key takes (5/5)

In the GUI	In pyfs	Verified on
Average an unsteady run over its last steps, per blade and at each azimuth	row keys <code>LAST_ITERS_AVG</code> or <code>LAST_REVS_AVG</code> , and <code>BLADES</code> ; pproc tables <code>[phase_locked]</code> , <code>[equations]</code> , <code>[glossary]</code> and <code>[names]</code> , and pproc key <code>blade_pattern</code>	none
Post-process a saved simulation again, without solving it again	row key <code>ADDITIONAL_PPROC</code> , naming a second pproc of sections, sectional loads and surface exports, which <code>pyfs-matrix post --additional-pproc</code> extracts from each recorded point's final <code>.fsm</code> with no solve	26.124, except one
Export a boundary-layer velocity profile	Separate pproc <code>[products]</code> flag <code>boundary_layer_velocity_profile</code> ; currently refused by name before execution. Integral products are not a substitute. The command has no positive unattended-build evidence: it opens a modal dialog on 26.122 and is broken on 26.124	none
Probe a surface property through an unsteady run	pproc table <code>[[surface_probes]]</code> , one per named point, parameter and frame	none

Source: `docs/gui-to-pyfs.md`, Basic post.

Basic post: not yet

- Trace streamlines, on the surface or through the field (GENERATE_ALL_SURFACE_STREAMLINES, NEW_OFF_BODY_STREAMLINE, EXPORT_ALL_SURFACE_STREAMLINES, EXPORT_ALL_OFF_BODY_STREAMLINES).
- Save a picture of the scene, coloured by a variable (SET_SCENE_CONTOUR, SAVE_SCENE_AS_IMAGE).
- Export the surface pressures as structural loads (EXPORT_SOLVER_ANALYSIS_PLOAD_BDF, verified 26.120 to 26.124).
- Import probe points from a file (PROBE_POINTS_IMPORT, verified 26.101 to 26.124).
- Animate a surface through an unsteady run (UNSTEADY_SOLVER_ANIMATION).

except one: one command of the line has no probe run on those builds; the page names it.

Source: docs/gui-to-pyfs.md, Basic post.

Part 7

The raw route

A step with no key: three ways a row still takes it

A `[[raw]]` line in the setup: every row naming the setup emits it.

```
[[raw]]  
command = "SET_TRAILING_EDGE_TYPE 3 RELAXED"  
before = "init"
```

A RAW record in the row's own cell, for that row alone:

```
RAW: {COMMAND: SET_TRAILING_EDGE_TYPE 3 RELAXED  
      / BEFORE: init}
```

A custom flag, for a command of one value; each row states `base_bending`: 12.5:

```
[[flags]]  
name = "base_bending"  
command = "SET_BASE_REGION_BENDING_ANGLE"
```

- Each line passes the checks a curated line passes: the command exists on the row's build, its arguments have the database's types, and it keeps the script's phase order. A line that fails is refused at plan, before any seat is spent.
- A raw line names what it acts on **by index**, a fact about one mesh file: keep it to the case it is written for.
- More than a few lines, such as a geometry from CAD: do it once in the GUI and save a `.fsm`. A **LEGACY** row runs a recipe of your own, in Python.

Source: docs/gui-to-pyfs.md, The raw route.

The whole session, in the order we work

1. **Geometry:** a `.fsm` prepared once in the GUI, or an OBJ in metres with its sidecar: names, unit, trailing edge by file (26.124).
2. **The study:** one row per position in `matriz.fs`, citing the reference, the setup (`farfield_layers = 5`) and the `pproc`.
3. `pyfs-matrix plan`: no seat spent; every refusal happens here.
4. `pyfs-matrix run`, then `collect` on the cluster; `--resume` for a matrix that grew, `--force-rerun` for a row that was wrong.
5. `pyfs-matrix post`: products under `post/`, and `post.log` read first.
6. **More later, with no solve:** `ADDITIONAL_PPROC` and `post --additional-pproc`, from each point's saved `.fsm` (26.124).

***Takeaway** The GUI is for what has no key yet. Everything with a key is a line in a file that the next person can read, plan and run again.*

The physics behind the steps, and where to read it

Step	The physics, and its origin
The surface mesh and its solve	a panel method: singularities on the surface panels, one boundary condition per panel [3, 4]
Trailing edges and wake nodes	the Kutta condition fixes the circulation at a sharp trailing edge, and the wake carries it downstream [4, 5]
Wake relaxation, wake termination	the wake is force-free: it is moved with the local velocity until it stops changing [4]
The compressibility model	the linearised subsonic correction of Prandtl and Glauert [6]
The boundary layer and its coupling	a viscous layer solved on the inviscid pressure and fed back to it [7, 8]
The separation models and criteria	turbulent separation by a pressure-gradient criterion, and a maximum-lift criterion [9, 10]
An actuator disc	the momentum theory of a propeller: a pressure jump and a swirl across a disc [11]
A body rate	the free stream seen from axes that turn with the aircraft [12]

References

- [1] Geovana Neves, *pyflightstream documentation*, page “From the GUI to pyfs”.
<https://nevesgeovana.github.io/pyflightstream/gui-to-pyfs/>
- [2] Geovana Neves, *pyflightstream: measurement records of the solver's behaviour*, 2026.
<https://github.com/nevesgeovana/pyflightstream>, concept DOI 10.5281/zenodo.21482924.
- [3] J. L. Hess, A. M. O. Smith, “Calculation of potential flow about arbitrary bodies”, *Progress in Aerospace Sciences*, 8, pp. 1-138, 1967.
- [4] J. Katz, A. Plotkin, *Low-Speed Aerodynamics*, 2nd ed., Cambridge University Press, Cambridge, 2001.
- [5] W. M. Kutta, “Auftriebskräfte in strömenden Flüssigkeiten”, *Illustrierte Aeronautische Mitteilungen*, 6, pp. 133-135, 1902.
- [6] H. Glauert, “The effect of compressibility on the lift of an aerofoil”, *Proceedings of the Royal Society of London, Series A*, 118(779), pp. 113-119, 1928.
- [7] H. Schlichting, K. Gersten, *Boundary-Layer Theory*, 9th ed., Springer, Berlin, 2017.

References (continued)

- [8] M. Drela, M. B. Giles, "Viscous-inviscid analysis of transonic and low Reynolds number airfoils", *AIAA Journal*, 25(10), pp. 1347-1355, 1987.
- [9] B. S. Stratford, "The prediction of separation of the turbulent boundary layer", *Journal of Fluid Mechanics*, 5(1), pp. 1-16, 1959.
- [10] W. O. Valarezo, V. D. Chin, "Method for the prediction of wing maximum lift", *Journal of Aircraft*, 31(1), pp. 103-109, 1994.
- [11] H. Glauert, *The Elements of Aerofoil and Airscrew Theory*, 2nd ed., Cambridge University Press, Cambridge, 1947.
- [12] B. Etkin, L. D. Reid, *Dynamics of Flight: Stability and Control*, 3rd ed., Wiley, New York, 1996.